CICLO FORMATIVO DE GRADO SUPERIOR

ADMINISTRACIÓN DE SISTEMAS INFORMÁTICOS EN RED

Proyecto de Fin de Ciclo

Diseño e Implementación de un Sistema de Monitorización en un Entorno de Producción Simulado

Pablo Dengra Maldonado

Tutor: José Repiso

Curso: CURSO 2024/2025

RESUMEN:

Este proyecto, titulado "Diseño e Implementación de un Sistema de Monitorización en un Entorno de Producción Simulado", surge de la necesidad de controlar y supervisar servicios informáticos de forma eficiente, tal como se haría en un entorno empresarial real. El trabajo se desarrolla dentro del contexto del ciclo formativo de Administración de Sistemas Informáticos en Red (ASIR), aplicando los conocimientos adquiridos a lo largo de la formación para construir una infraestructura controlada y observable.

El objetivo fundamental ha sido crear una solución de monitorización capaz de recoger métricas y logs de diferentes servicios clave (servidor web Apache, base de datos MySQL y servicios de red), permitiendo visualizar su estado en tiempo real, generar alertas y analizar posibles incidencias. Para esta finalidad se han utilizado herramientas como Prometheus, Grafana, Loki, mysqld_exporter y nginx_exporter, además de automatizaciones mediante scripts. El proyecto se ha desarrollado por fases: diseño, despliegue, configuración, pruebas y validación. Como resultado, se ha conseguido una infraestructura funcional, capaz de detectar anomalías, mostrar visualizaciones claras por servicio y enviar alertas automáticas, lo que demuestra la eficacia de las herramientas usadas y la importancia de la monitorización proactiva en sistemas reales.

ABSTRACT:

This project, titled "Design and Implementation of a Monitoring System in a Simulated Production Environment", arises from the need to efficiently control and supervise IT services, just as it would be done in a real business environment. The work is carried out within the framework of the Advanced Vocational Training Course in Administration of Networked Computer Systems (ASIR), applying the knowledge acquired throughout the program to build a controlled and observable infrastructure.

The main objective has been to create a monitoring solution capable of collecting metrics and logs from various key services (Apache web server, MySQL database,

and network services), allowing real-time status visualization, alert generation, and incident analysis. Tools such as Prometheus, Grafana, Loki, mysqld_exporter, and nginx_exporter have been used for this purpose, along with automation through scripts. The project has been developed in several phases: design, deployment, configuration, testing, and validation. As a result, a functional infrastructure has been achieved, capable of detecting anomalies, displaying clear service-specific visualizations, and sending automatic alerts, demonstrating the effectiveness of the tools used and the importance of proactive monitoring in real systems.

AGRADECIMIENTOS

Quiero expresar mi más sincero agradecimiento al tutor del proyecto, por su orientación, dedicación y valiosas sugerencias a lo largo de todo el desarrollo del trabajo. Su experiencia y apoyo han sido fundamentales para alcanzar los objetivos propuestos.

Agradezco también a los profesores y docentes que han impartido asignaturas relacionadas con este proyecto, por compartir sus conocimientos y por fomentar un aprendizaje práctico que ha sido clave para su realización de este trabajo.

A Digitech, por facilitar los medios, recursos y un entorno adecuado para desarrollar este proyecto con garantías.

Y, por supuesto, a mi familia, por el apoyo incondicional, comprensión, paciencia y aguante durante todo el proceso.

Índice

Contenido

1. Introducción 4
1.1 Objetivos 4
1.2 Contextualización 5
1.2 Justificación 5
2. Desarrollo del proyecto 5
2.1 Metodología de trabajo 6
2.2 Planificación 7
2.3 Herramientas y tecnologías utilizadas 9
2.4 Desarrollo 11
2.5 Pruebas y resultado 23
3. Conclusiones y trabajos futuros 29

1. Introducción

En los entornos con infraestructuras informáticas, la monitorización de sistemas y servicios se ha convertido en una tarea esencial para garantizar el rendimiento, la disponibilidad y la seguridad de las infraestructuras. Los sistemas de monitorización que están bien diseñados, permiten detectar problemas, antes de afectar a los usuarios finales, y conseguir así tomar decisiones basadas en datos en tiempo real, para facilitar tareas de mantenimiento preventivo.

Este trabajo tiene la finalidad de implementar un diseño e implementación de un sistema de monitorización integral para un entorno de producción simulado que incluye varios servicios fundamentales: un servidor web, una base de datos MySQL y servicios de red básicos. Mediante la utilización de herramientas especializadas como Prometheus, Grafana y Loki, se recogerán métricas y logs, permitiendo la visualización, análisis y generación de alertas automatizadas.

1.1 Objetivos

El objetivo fundamental de este trabajo es la instalación y puesta en marcha de un sistema de monitorización para un entorno simulado, que permita visualizar el estado y comportamiento de varios servicios de manera centralizada.

Objetivos principales:

- Implementar la recolección de métricas mediante Prometheus.
- Configurar Grafana para visualizar datos en tiempo real.
- Recopilar logs del sistema y servicios mediante Loki.
- Desplegar exporters de Apache (NGINX exporter), MySQL (mysqld_exporter) y otros servicios.
- Incorporar alertas automáticas basadas en umbrales críticos.
- Integrar automatizaciones de parte de la infraestructura mediante scripts.

1.2 Contextualización

Este proyecto ha sido elaborado, teniendo presente el contexto del Grado Superior en Administración de Sistemas Informáticos en Red (ASIR), donde uno de los puntos relevantes son el conocimiento y gestión de infraestructuras IT. En los últimos años, la observación de datos en tiempo real ha ido ganado protagonismo como estrategia clave para la administración de sistemas, especialmente en entornos de alta disponibilidad o servicios críticos.

Al poner en marcha un entorno de producción realista, este proyecto permite aplicar en un entorno controlado los conocimientos adquiridos sobre sistemas, redes, seguridad y administración de servidores, adaptándolos a una práctica profesional con herramientas que pueden ser utilizadas en un entorno empresarial real.

1.2 Justificación

Este proyecto se ha elegido por la necesidad de dar soluciones de monitorización adaptadas a entornos diversos y escalables. La utilización de herramientas como Prometheus, Grafana y Loki ofrecen un ecosistema robusto y flexible que permite integrar diferentes servicios y obtener una visibilidad completa del sistema.

A su vez, este trabajo busca fortalecer competencias clave como la automatización, la interpretación de métricas y logs, y la capacidad de anticiparse a posibles incidencias mediante sistemas de alerta. Estas habilidades resultan altamente demandadas en el sector tecnológico actual y son un complemento ideal para la formación académica del ciclo.

2. Desarrollo del proyecto

Esta parte consiste en describir el proceso que se ha llevado a cabo para implementar el sistema de monitorización, incluyendo la planificación, instalación, configuración y puesta en marcha de los distintos componentes que se han utilizado.

2.1 Metodología de trabajo

La metodología llevada a cabo en este proyecto se ha basado en un enfoque desarrollado paso a paso. Se han definido varias fases para abordar cada parte del sistema de forma independiente y escalonada.

- **Fase de planificación:** Se identificaron los servicios a monitorizar (Apache, MySQL, servicios de red) y las herramientas más adecuadas para ello (**Prometheus, Grafana, Loki, exporters**).

- **Fase de implementación:**

  a) Se lleva a cabo la instalación y configuración de Prometheus como motor de recolección de métricas.

  b) Despliegue de mysqld_exporter para monitorizar MySQL y nginx_exporter adaptado para Apache.

  c) Se lleva a cabo la configuración de Loki junto a Promtail para la recogida de logs del sistema y servicios.

  d) Se realiza la integración de todos los datos en Grafana mediante dashboards específicos por servicio.

- **Fase de alerta:** Se crean reglas de alerta en Prometheus basadas en umbrales (uso de CPU, RAM, errores en logs, etc.) y configuración de notificaciones (correo o Slack).

- **Fase de automatización:** Se crean scripts de bash y python para el despliegue automático de los servicios y configuración del sistema.

- **Fase de pruebas y validación:** Se realiza simulación de fallos, análisis de los dashboards, revisión de alertas y ajustes en las configuraciones.

Gracias a esta forma de realizar el trabajo, se ha podido avanzar paso a paso, y así, comprobar que todo funciona bien antes de seguir con lo siguiente.

2.2 Planificación

A continuación, paso a describir las fases del proyecto:

2.2.1 INVESTIGACIÓN

El proyecto se ha comenzado llevando a cabo una investigación técnica sobre herramientas de monitorización Open Source orientadas a infraestructuras pequeñas o domésticas, con el objetivo de montar un entorno funcional sobre una Raspberry Pi.

Se han consultado las siguientes fuentes:

- Documentación oficial de Prometheus, Grafana y Loki.
- Manuales y foros sobre exporters específicos (MySQL, NGINX, Apache…).
- Guías de configuración de Alertmanager y métodos de envío de alertas por Telegram.
- Blogs técnicos y experiencias de usuarios en entornos similares.

A su vez, se han analizado las necesidades que pueden surgir en entornos como redes domésticas, centros educativos o pequeñas empresas, teniendo en cuenta la necesidad que existe de vigilar el estado de los servicios, recibir alertas automáticas si algo falla y de tener un historial gráfico de métricas y logs.

Los conocimientos que se han adquirido para llevar a cabo el proyecto incluyen:

- Administración básica de sistemas Linux.
- Uso de servicios en segundo plano (systemd).
- Configuración de red y servicios web (Apache/Nginx).
- Shell scripting para tareas automatizadas.
- Fundamentos de redes y monitorización.

2.2.2 DISEÑO

Antes de poner en marcha este sistema de monitorización, se ha diseñado una arquitectura sencilla, capaz de ejecutarse en una Raspberry Pi con recursos limitados.

Elementos del sistema:

- **Servidor central de monitorización:** Raspberry Pi con SO Linux (Raspberry Pi OS) con 16Gb de RAM.
- **Servicios monitorizados:** Prometheus, Grafana, Loki, Apache, NGINX, MySQL.
- **Exporters instalados:** mysqld_exporter, node_exporter.
- **Alertas:** Enviadas vía Telegram (API Bot).
- **Automatización:**
  - Script bash de mantenimiento que revisa el estado de los servicios, elimina logs antiguos y alerta en caso de fallos.
  - Script python para recibir alertas enviadas desde Alertmanager (de Prometheus) vía HTTP POST y reenviarlas como mensajes formateados a un chat de Telegram usando un bot.

Decisiones clave:

- Se optó por Prometheus y Grafana por su potencia, integración y compatibilidad con exporters.
- Se cogió Loki como sistema de logs por la integración nativa que tiene con Grafana.
- Se diseñó varios scripts, uno en Bash y otro en Python, para automatizar tareas de mantenimiento básicas, aprovechando los recursos del sistema sin necesidad de herramientas externas.
- Se utilizó Telegram para alertas por su inmediatez y simplicidad en la configuración con bots.

Herramientas utilizadas para el diseño:

Se ha realizado una planificación y seguimiento del trabajo mediante una tabla cronograma.

2.2.3 CRONOGRAMA Y ORGANIZACIÓN TEMPORAL

Se ha organizado el proyecto en fases por semana, teniendo en cuenta la complejidad de cada parte y la carga de trabajo. El calendario general ha sido el siguiente:

Tabla 1. Cronograma de fases por semana.

Semana	Actividad
1	Investigación, análisis de necesidades, elección de herramientas
2	Instalación y configuración de la Raspberry Pi, red e IP estática
3	Instalación de Prometheus, Exporters y pruebas de recolección de métricas
4	Instalación de Grafana y creación de dashboards personalizados
5	Instalación de Loki y envío de logs
6	Desarrollo de scripts de mantenimiento y automatización
7	Configuración de alertas por Telegram
8	Pruebas finales, validación del sistema y redacción de la memoria

2.3 Herramientas y tecnologías utilizadas

En este proyecto se han utilizado diferentes herramientas y tecnologías con el objetivo de crear un sistema completo de monitorización, registro y alerta en una red de servicios.

A continuación, se detallan las principales herramientas:

Sistema base.

- **Raspberry Pi 5 (16GB RAM)**: Equipo principal que actúa como servidor de monitorización. Se eligió por su bajo consumo y suficiente capacidad para ejecutar los servicios necesarios.
- **Sistema operativo**: Raspberry Pi OS Lite (basado en Debian), por su ligereza y compatibilidad con herramientas de administración.

Monitorización y métricas.

- **Prometheus**: Herramienta principal de monitorización que recopila métricas a través de endpoints expuestos por los servicios o exporters.

- **Exporters utilizados**:
  - node_exporter: Información del sistema (CPU, RAM, discos…).
  - mysqld_exporter: Métricas del servidor MySQL.
  - nginx_exporter: para ver tráfico y estado de los servidores web.
- **Grafana**: Plataforma de visualización de datos utilizada para crear dashboards personalizados con los datos recogidos por Prometheus.

Logs y registro.

- **Loki**: Sistema centralizado de logs, diseñado para integrarse con Grafana. Permite almacenar y consultar logs de servicios como Apache, NGINX o el propio sistema.
- **Promtail**: Agente que recoge los logs locales y los envía a Loki.

Alertas y notificaciones.

- **Telegram Bot API**: Se creó un bot de Telegram que envía notificaciones automáticas al detectar errores o fallos en los servicios monitorizados.
- **Alertmanager**: Gestopma las alertas que genera Prometheus cuando detecta que algo no va bien en loa sistemas o servicios.

Automatización.

- **Bash scripting**: Se desarrolló un script de mantenimiento automático (mantenimiento_monitorizacion.sh) que:
  - Comprueba el estado de los servicios.
  - Reinicia aquellos que estén caídos.
  - Elimina logs antiguos de Loki.
  - Envía alertas por Telegram si un exporter deja de responder.
- **Script python:** Para la recepción de las alertas de Alertmanager y poder enviarlas a Telegram en caso de alertas del sistema.
- **Cron (crontab)**: Programador de tareas utilizado para ejecutar periódicamente el script de mantenimiento de forma automatizada.

Otras herramientas.

- **curl**: Herramienta de línea de comandos para realizar peticiones HTTP, muy útil para probar endpoints de exporters o enviar mensajes a la API de Telegram.
- **VS Code**: Editores de texto utilizados para la edición de scripts y ficheros de configuración.

Estas herramientas han sido seleccionadas por su fiabilidad, facilidad de integración y porque muchas de ellas son software libre, lo que permite estudiar su funcionamiento en profundidad y adaptarlas a distintos entornos reales. Además, han demostrado ser suficientemente ligeras como para ejecutarse correctamente en una Raspberry Pi.

2.4 Desarrollo

El proyecto se ha dividido en varias fases prácticas que han permitido construir un sistema completo de monitorización en una Raspberry Pi con una dirección IP estática.

A lo largo del proceso, se han instalado y configurado distintos servicios que permiten observar, analizar y mantener el sistema de forma automatizada.

2.4.1 Instalación del entorno base

Se ha utilizado una Raspberry Pi con sistema operativo Raspberry Pi OS Lite como servidor principal del proyecto. Se le asignó una IP estática y se configuró el acceso remoto mediante SSH para facilitar la gestión del sistema.

Figura 1. Configuración de red.

```
pidenguer@raspberrypi:~ $ nmcli device show eth0
GENERAL.DEVICE:                         eth0
GENERAL.TYPE:                           ethernet
GENERAL.HWADDR:                         2C:CF:67:8C:3D:EC
GENERAL.MTU:                            1500
GENERAL.STATE:                          100 (conectado)
GENERAL.CONNECTION:                     Conexión cableada 1
GENERAL.CON-PATH:                       /org/freedesktop/NetworkManager/ActiveConnection/20
WIRED-PROPERTIES.CARRIER:               activado
IP4.ADDRESS[1]:                         192.168.1.24/24
IP4.GATEWAY:                            192.168.1.1
IP4.ROUTE[1]:                           dst = 0.0.0.0/0, nh = 192.168.1.1, mt = 100
IP4.ROUTE[2]:                           dst = 192.168.1.0/24, nh = 0.0.0.0, mt = 100
IP6.ADDRESS[1]:                         fe80::25b8:886e:775d:cf04/64
IP6.GATEWAY:                            --
IP6.ROUTE[1]:                           dst = fe80::/64, nh = ::, mt = 1024
```

2.4.2. Prometheus

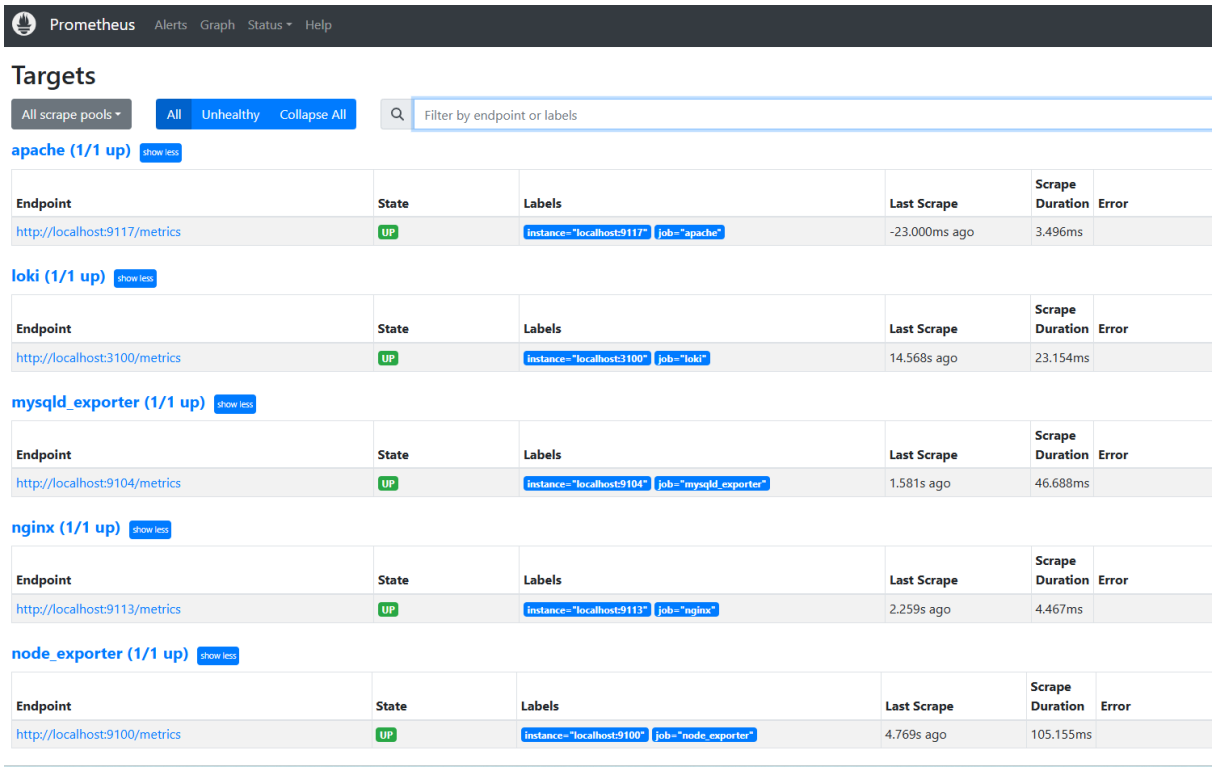
Prometheus es el núcleo de la recogida de métricas. Yo lo utilizo principalmente para **controlar el estado y rendimiento de los servicios** desplegados en la Raspberry Pi y en otros equipos conectados a la red.

He configurado Prometheus para que recoja datos de los siguientes *exporters*:

- node_exporter: métricas del sistema (CPU, RAM, disco, etc.).
- mysqld_exporter: métricas del servidor MySQL.
- nginx y apache: a través de endpoints de estado.

Además, le he añadido reglas personalizadas para detectar si un servicio no está enviando métricas o si hay sobrecarga en el sistema.

Figura 2. Imagen donde se ve el estado de los targets.



2.4.3. Grafana

Grafana es un software que se ha incorporado para la monitorización de métricas y logs. Es la herramienta que utilizo para visualizar todos los datos recogidos.

Por ejemplo, tengo un panel donde aparecen las métricas del uso de CPU, RAM, tráfico de red o disponibilidad de servicios en tiempo real. También me permite analizar tendencias y tomar decisiones sobre posibles ajustes o mantenimiento.
Además, se ha utilizado Loki como herramienta para la recolección de logs, permitiendo centralizar y filtrar eventos del sistema.

Figura 3. Dashboard realizado en Grafana.

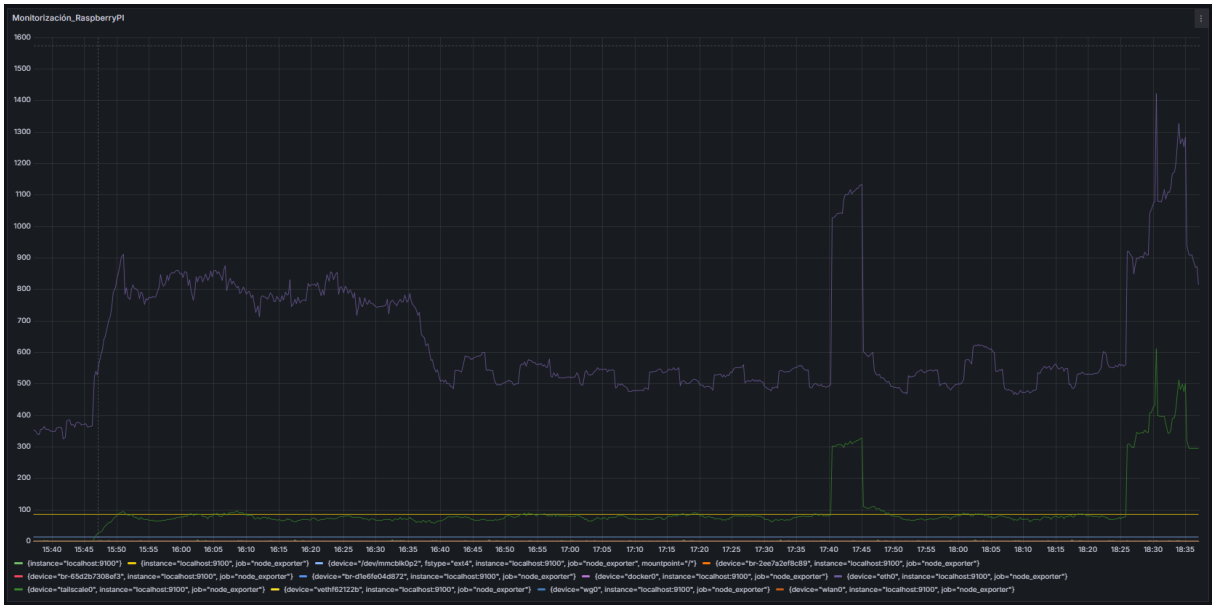


Figura 4. Códigos de métricas para ver el uso CPU y memoría libre/uso.

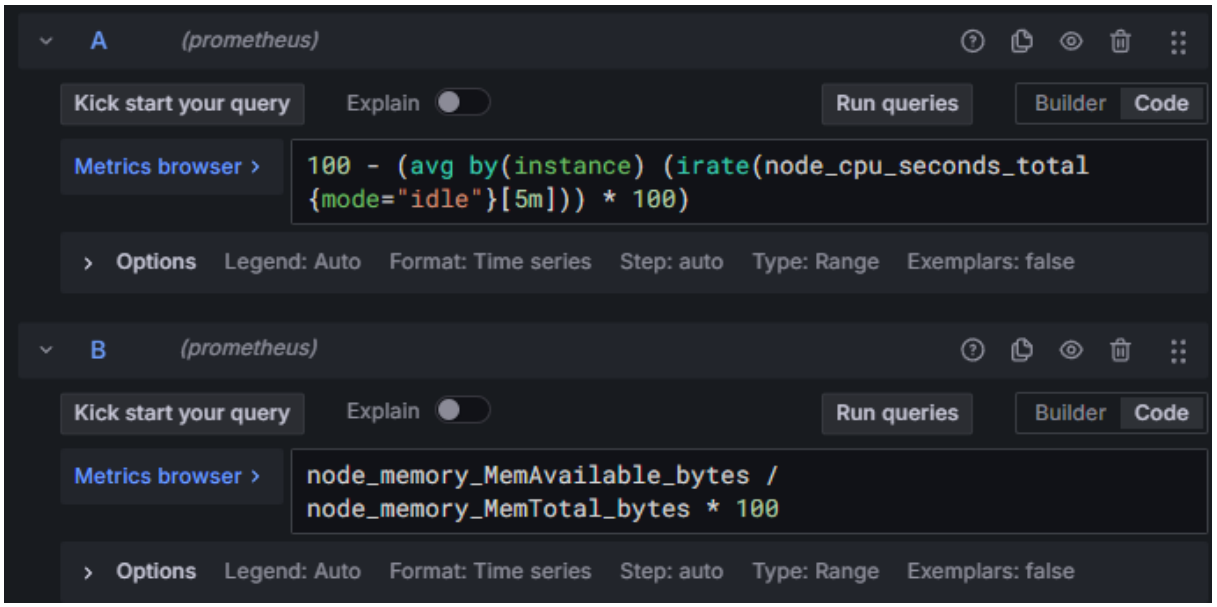
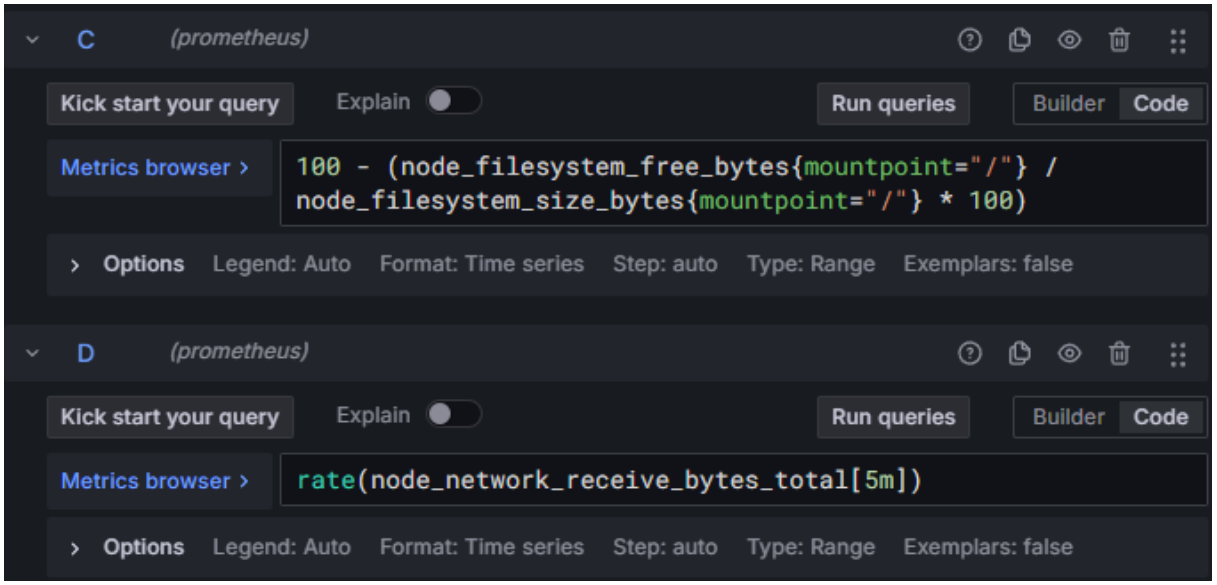


Figura 5. Código de métricas para ver el uso del disco y tráfico de red.



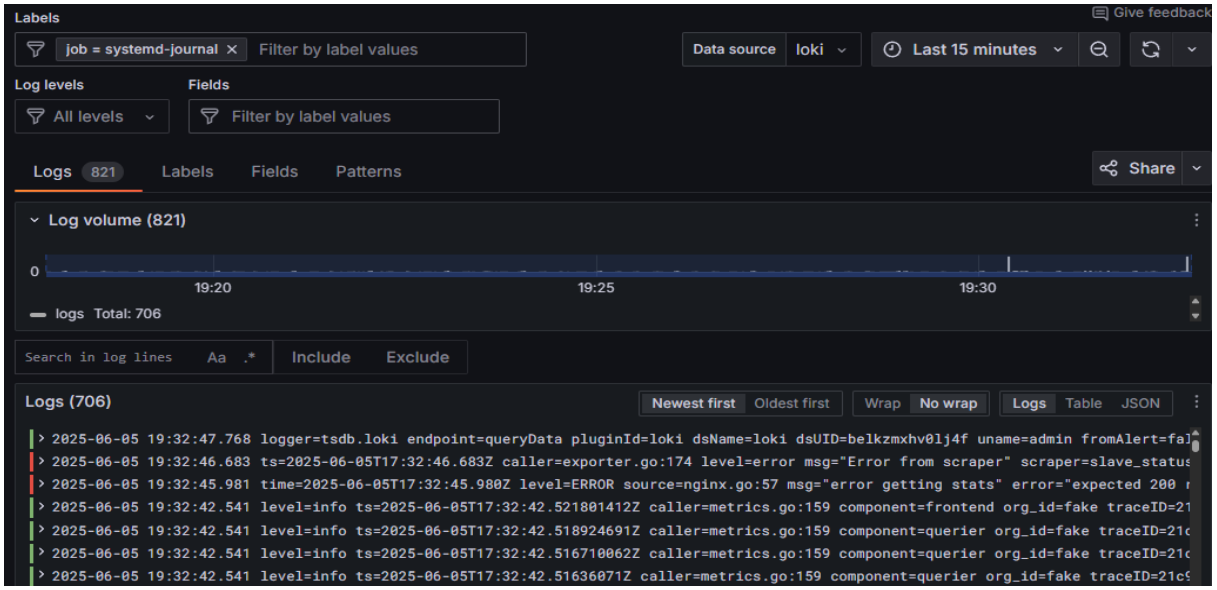
2.4.4. Integración de Loki.

Loki lo uso para gestionar los logs de forma centralizada. Antes debía entrar en cada servidor o buscar manualmente en los archivos de /var/log, pero ahora todos los logs están en un solo lugar y accesibles desde Grafana.

Con Loki puedo, por ejemplo:

- Buscar errores recientes en nginx o apache.
- Detectar si un servicio ha caído por algún motivo concreto (por ejemplo, fallo de conexión a base de datos).

Figura 6. Filtrado de logs proveniente de systemd-journal.



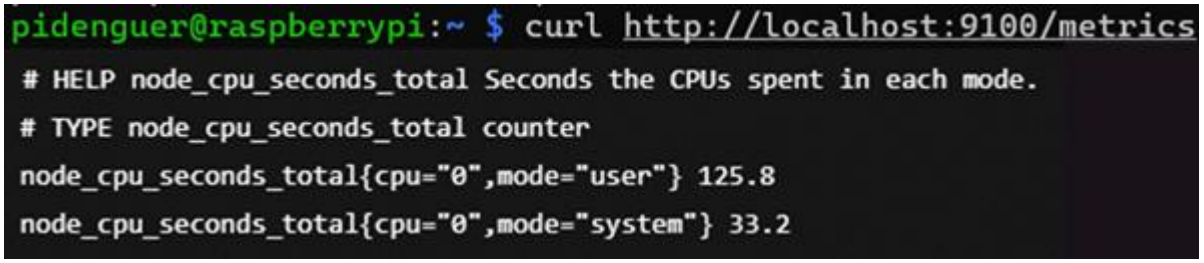
Permite mostrar logs del sistema recogidos desde el systemd journal, es decir servicios iniciados o detenidos, errores de procesos del sistema, mensajes del Kernel, fallos de servicios de nginx.service, etc…

2.4.5. Exporters implementados

Se han instalado y configurado varios exporters para que Prometheus recoja datos de distintas fuentes:

- **mysqld_exporter:** para monitorear el estado de MySQL.
- **node_exporter:** para estadísticas del sistema como CPU, memoria y red.
- **nginx_exporter:** para ver tráfico y estado del servidor web.

Figura 7. Visualización desde terminal métricas de node_exporter.



Muestra las métricas de los segundos totales que la CPU ha pasado en cierto modo, en este caso “user o system”.

Figura 8. Visualización desde terminal métricas de mysql_exporter.

```
pidenguer@raspberrypi:~ $ curl http://localhost:9104/metrics
# HELP mysql_global_status_threads_connected Number of currently open connections.
mysql_global_status_threads_connected 6
```

Esta métrica, muestra el número de conexiones que hay abiertas, en este caso, hay seis clientes conectados a la base de datos.

Figura 9. Visualización desde terminal de métricas nginx_exporter.

```
pidenguer@raspberrypi:~ $ curl http://localhost:9113/metrics
# HELP nginx_connections_active Active connections
nginx_connections_active 2
```

Esta métrica, muestra las conexiones activas que hay en nginx, en este caso, hay dos conexiones.

2.4.6. Configuración de Alertmanager

Como parte del sistema de monitorización, se ha implementado Alertmanager, el componente oficial de Prometheus encargado de gestionar las alertas. El objetivo fundamental es recibir notificaciones cuando alguna métrica está fuera de un rango específico o un servicio deja de responder, y enviarlas al bot de mensajería Telegram.

En este proyecto Alertmanager está configurado para recibir alertas, que se detallan a continuación:

- Alerta si el uso de la CPU ha sido superior al 80 % durante más de 5 minutos.
- Si MySQL exporter no responde desde más de 5 minutos.
- Si el uso de memoria RAM es mayor de un 90%.
- SI disco duro tiene más de un 90% usado.
- Si hay más de 100 conexiones activas detectadas.
- SI nginx exporter ha dejado de estar disponible.
- Si existen errores HTTP 5xx elevados.

Estas reglas se incluyen en el fichero principal de configuración de Prometheus.

Figura 10. Reglas de alerta del archivo alert.rules.yml.

```
  GNU nano 7.2                                        /home/pidenguer/mi-proyecto/alert.rules.yml
groups:
  - name: system_alerts
    rules:
      - alert: HighCPUUsage
        expr: 100 - (avg by(instance)(irate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "El uso de CPU está por encima del 80% en {{ $labels.instance }}"
          description: "El uso de CPU ha sido superior al 80% durante más de 5 minutos."

      - alert: MySQLDown
        expr: mysql_up == 0
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "MySQL está caído en {{ $labels.instance }}"
          description: "El exportador MySQL no responde desde hace más de 5 minutos."

      - alert: HighMemoryUsage
        expr: (node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / node_memory_MemTotal_bytes > 0.9
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Uso de RAM alto en {{ $labels.instance }}"
          description: "Uso de memoria RAM > 90%."

      - alert: LowDiskSpace
        expr: (node_filesystem_size_bytes{fstype!="tmpfs",fstype!="overlay"} - node_filesystem_free_bytes{fstype!="tmpfs",fstype!="overlay"}) / node_filesystem_siz
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Espacio en disco bajo en {{ $labels.instance }}"
          description: "Más del 90% del disco usado."

      - alert: HighMySQLConnections
        expr: mysql_global_status_threads_connected > 100
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Muchas conexiones a MySQL en {{ $labels.instance }}"
          description: "Más de 100 conexiones activas detectadas."

      - alert: NGINXDown
        expr: nginx_up == 0
        for: 2m
        labels:
         severity: critical
        annotations:
         summary: "NGINX no responde en {{ $labels.instance }}"
         description: "El exportador de NGINX ha dejado de estar disponible."

      - alert: HighHTTP5xxRate
        expr: sum(rate(apache_http_requests_total{status=~"5..*"}[5m])) > 5
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "Errores HTTP 5xx elevados en {{ $labels.instance }}"
```

También se ha configurado el archivo **alertmanager.yml** para incluir el Webhook de Telegram. Cuyo objetivo es permitir que cuando una alerta se activa, se envíe automáticamente un mensaje de aviso al canal configurado.

Figura 11. Visualización archivo alertmanager.yml.

```
pidenguer@raspberrypi:/opt/mis-scripts $ cat /opt/alertmanager/alertmanager.yml
global:
  # Aquí otras configuraciones globales si tienes

route:
  receiver: 'telegram-webhook'

receivers:
  - name: 'telegram-webhook'
    webhook_configs:
      - url: 'http://192.168.1.24:5000/alert'
```

Figura 12. Visualización alertas archivo alert.rules.yml.

```
  GNU nano 7.2                                        /home/pidenguer/mi-proyecto/alert.rules.yml
groups:
  - name: system_alerts
    rules:
      - alert: HighCPUUsage
        expr: 100 - (avg by(instance)(irate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "El uso de CPU está por encima del 80% en {{ $labels.instance }}"
          description: "El uso de CPU ha sido superior al 80% durante más de 5 minutos."

      - alert: MySQLDown
        expr: mysql_up == 0
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "MySQL está caído en {{ $labels.instance }}"
          description: "El exportador MySQL no responde desde hace más de 5 minutos."

      - alert: HighMemoryUsage
        expr: (node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / node_memory_MemTotal_bytes > 0.9
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Uso de RAM alto en {{ $labels.instance }}"
          description: "Uso de memoria RAM > 90%."
```

Figura 13. Visualización de la alerta en Alertmanager.

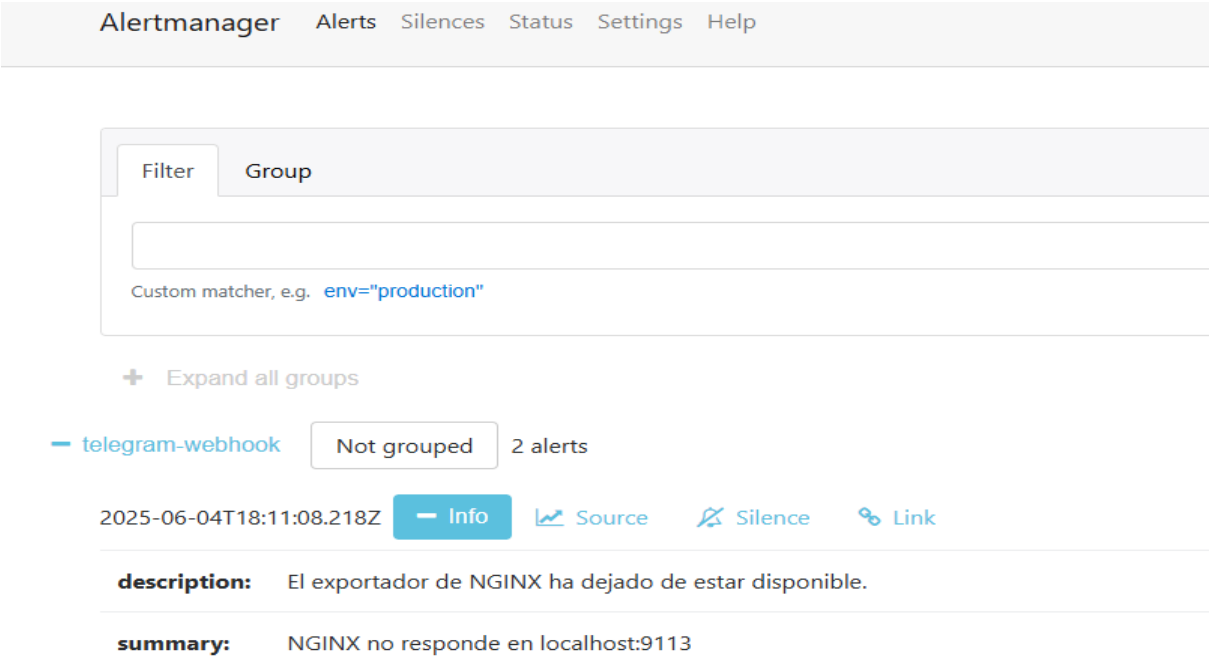
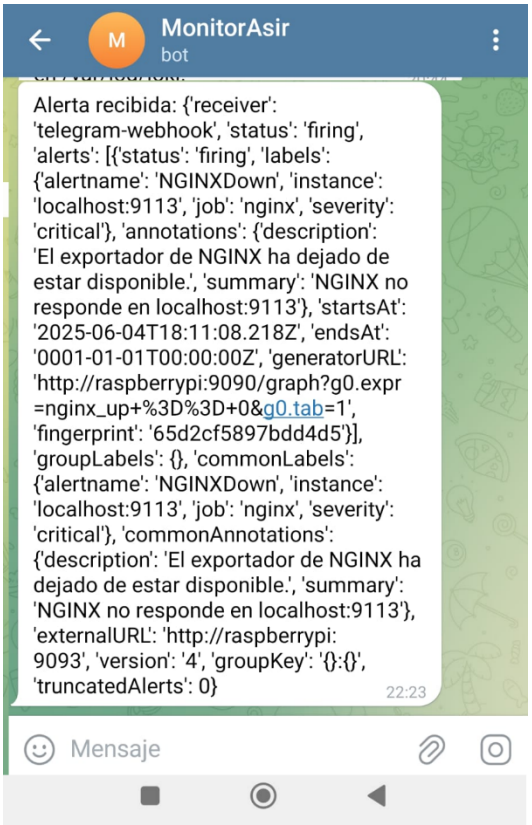


Figura 14. Alerta de Telegram de Alertmanager.



Con esta implementación, el sistema no solo recopila y visualiza métricas, sino que también actúa de forma proactiva ante fallos mediante alertas automatizadas.

2.4.7. Automatización de mantenimiento

Se ha desarrollado un script en Bash llamado mantenimiento_monitorizacion.sh que realiza tareas automáticas:

- Reinicia servicios en caso de fallo (Prometheus, Grafana, Loki, Nginx, Apache…).
- Limpia logs antiguos en la carpeta de Loki.
- Verifica si los exporters están respondiendo correctamente mediante curl.
- Envía notificaciones por Telegram si algún servicio falla.

Este script se ejecuta de forma manual o automática mediante una entrada en crontab.

Figura 15-1. Visualización del script de mantenimiento del sistema.

```bash
#!/bin/bash
# Script para mantenimiento de monitorización:
# - Reinicia servicios que fallen o se cuelguen
# - Limpia logs antiguos en Loki (mayores a 7 días)
# - Verifica el estado de exporters y alerta si algo no responde

# Servicios a controlar
SERVICIOS=("prometheus" "grafana-server" "loki" "mysqld_exporter" "apache2" "nginx")

# Configuración de Telegram
TELEGRAM_BOT_TOKEN="8132527691:AAHPiZD9IOODSrYgSJcQkgC_EBxV9Enkx3k"
TELEGRAM_CHAT_ID="5402866448"

# Función para enviar mensaje a Telegram
enviar_alerta_telegram() {
  local MENSAJE="$1"
  curl -s -X POST "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/sendMessage" \
    -d chat_id="${TELEGRAM_CHAT_ID}" \
    -d text="${MENSAJE}" > /dev/null
}

# Función para reiniciar servicio si no está activo
check_and_restart_service() {
  local servicio=$1
  systemctl is-active --quiet "$servicio"
  if [ $? -ne 0 ]; then
    echo "$(date): $servicio no está activo, reiniciando..."
    systemctl restart "$servicio"
    if [ $? -eq 0 ]; then
      echo "$(date): $servicio reiniciado correctamente."
      enviar_alerta_telegram "⚠️ *$servicio* no estaba activo. Se ha reiniciado correctamente."
    else
      echo "$(date): Error al reiniciar $servicio."
      enviar_alerta_telegram "❌ Error al intentar reiniciar *$servicio*. Revisa manualmente."
    fi
  else
    echo "$(date): $servicio está activo."
  fi
}

# Limpiar logs antiguos de Loki (ajusta la ruta si usas otra)
limpiar_logs_loki() {
  local ruta_logs="/var/log/loki"
  echo "$(date): Limpiando logs de Loki en $ruta_logs mayores a 7 días..."
  find "$ruta_logs" -type f -mtime +7 -exec rm -f {} \;
  enviar_alerta_telegram "🧹 Se han limpiado logs antiguos de *Loki* en $ruta_logs."
}

# Verificar estado de exporters (puertos o endpoints comunes)
check_exporter() {
  local nombre=$1
  local url=$2
  # Intentar conectar al endpoint HTTP
  if curl -s --head --request GET "$url" | head -n 1 | grep "200 OK" > /dev/null; then
    echo "$(date): $nombre responde correctamente."
  else
    echo "$(date): ALERTA: $nombre no responde en $url"
    enviar_alerta_telegram "🚨 *$nombre* no responde en: $url"
  fi
}

echo "===== INICIO mantenimiento_monitorizacion.sh ====="

# Revisar servicios
for servicio in "${SERVICIOS[@]}"; do
  check_and_restart_service "$servicio"
done
```

Figura 15-2. Visualización del script de mantenimiento del sistema.

```
# Limpiar logs
limpiar_logs_loki

# Verificar exporters
check_exporter "mysqld_exporter" "http://localhost:9104/metrics"
check_exporter "apache2" "http://localhost/server-status?auto"
check_exporter "nginx" "http://localhost/nginx_status"

echo "===== FIN mantenimiento_monitorizacion.sh ====="
```

Figura 16. Ejecución del script de mantenimiento del sistema.

```
pidenguer@raspberrypi:~ $ /opt/mis-scripts/mantenimiento_monitorizacion.sh
===== INICIO mantenimiento_monitorizacion.sh =====
jue 05 jun 2025 20:44:49 CEST: prometheus está activo.
jue 05 jun 2025 20:44:49 CEST: grafana-server está activo.
jue 05 jun 2025 20:44:49 CEST: loki está activo.
jue 05 jun 2025 20:44:49 CEST: mysqld_exporter está activo.
jue 05 jun 2025 20:44:49 CEST: apache2 está activo.
jue 05 jun 2025 20:44:49 CEST: nginx está activo.
jue 05 jun 2025 20:44:49 CEST: Limpiando logs de Loki en /var/log/loki mayores a 7 días...
jue 05 jun 2025 20:44:50 CEST: mysqld_exporter responde correctamente.
jue 05 jun 2025 20:44:50 CEST: ALERTA: apache2 no responde en http://localhost/server-status?auto
jue 05 jun 2025 20:44:50 CEST: nginx responde correctamente.
===== FIN mantenimiento_monitorizacion.sh =====
```

Realizo automatización con cron entrando en crontab –e. El script se ejecutará a las 2:30h de la madrugada todos los días.

Figura 17. Visualización de script automatizado.

```
pidenguer@raspberrypi:~ $ crontab -l
30 2 * * * /opt/mis-scripts/mantenimiento_monitorizacion.sh >> /opt/mis-scripts/log_mantenimiento.log 2>&1
```

2.4.8. Alertas con Telegram

Se ha creado un bot de Telegram para recibir notificaciones sobre el estado del sistema. Se configuró una llamada curl a la API de Telegram dentro del script para enviar mensajes en caso de error o caída de servicios.

Figura 18. Chat Telegram con bot funcionando.

Con este conjunto de herramientas integradas se ha conseguido obtener una plataforma de monitorización funcional, automatizada que avisa de si hay algún problema en el sistema, alertándome desde Telegram de si hay algún problema en el sistema.

2.5 Pruebas y resultado

Una vez implementado el sistema de monitorización y automatización, se ha procedido a realizar una serie de pruebas para comprobar su correcto funcionamiento. Estas pruebas se han centrado en verificar la recogida de métricas, el comportamiento de los dashboards, la generación de alertas y la respuesta ante fallos o incidencias simuladas.

2.5.1 Pruebas realizadas

1. Prueba de recolección de métricas

Se han realizado pruebas con Prometheus para verificar el resultado obtenido con los exporter instalados: node_exporter, mysqld_exporter, nginx y apache2.

A su vez, se ha comprobado que estas métricas se recogen correctamente en Grafana.

Figura 19. Dashboard de Grafana mostrando métricas (CPU, RAM, tráfico, etc.).

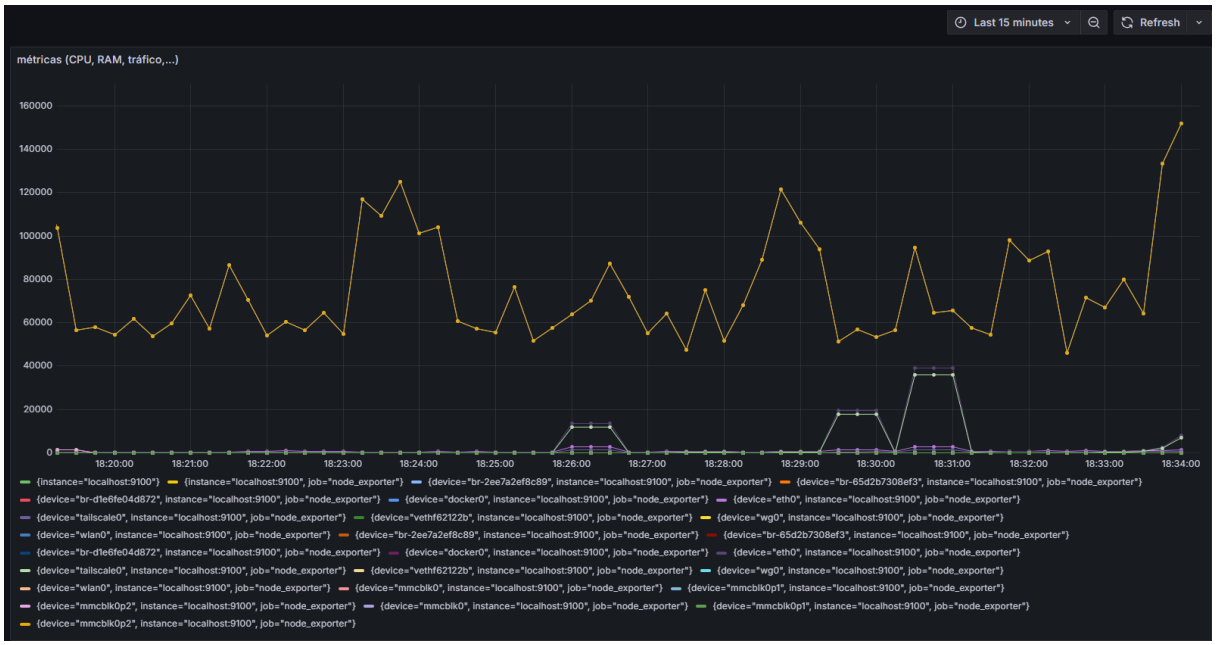
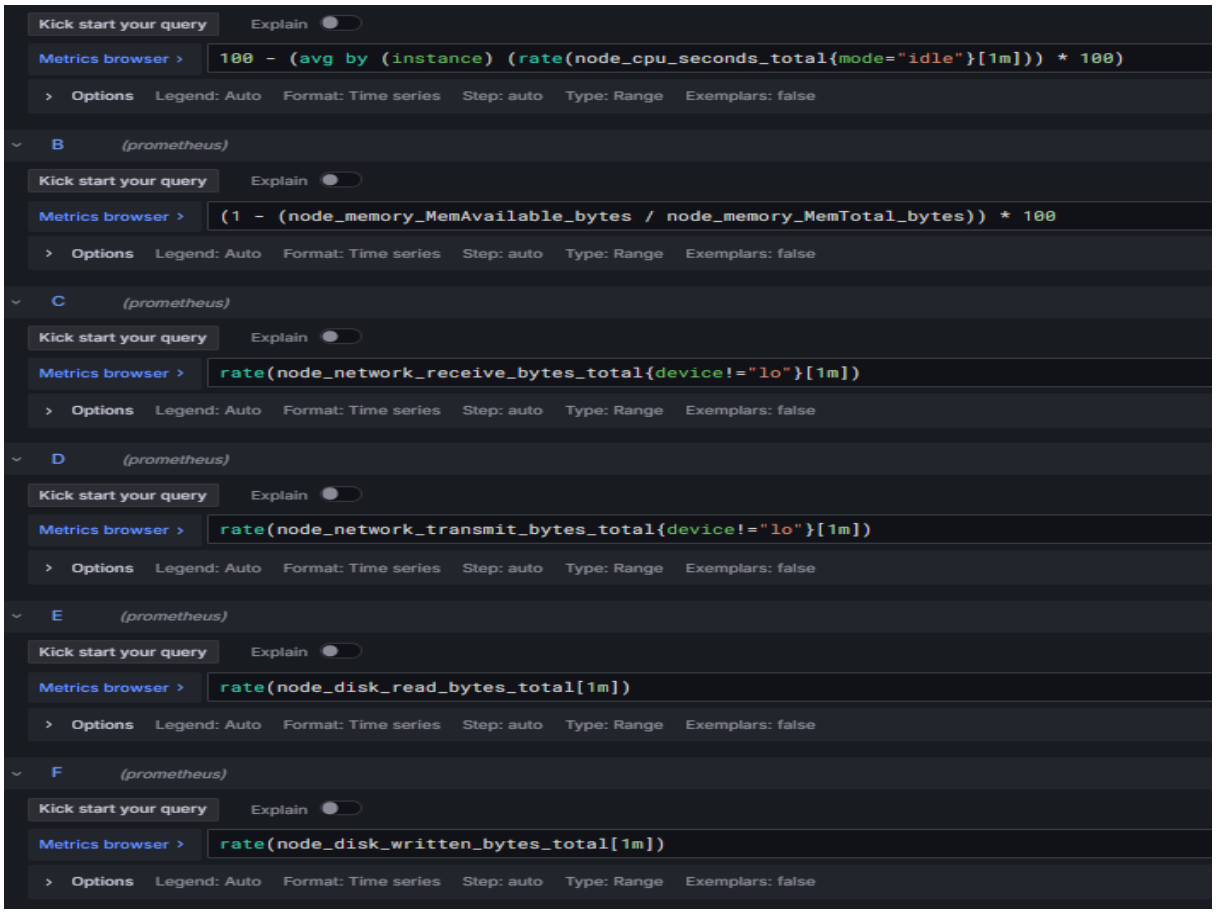


Figura 20. Consultas PromQL para dashboard en Grafana.



2. Prueba de visualización de logs con Loki

Se han realiado pruebas con logs desde Grafana usando Loki, filtrando por nivel de error, servicio o palabras clave. Se ha probado con logs de **mysql y systemd**.

Se ha realizado una búsqueda por **{filename="/var/log/mysql/error.log"} |= “”**, para filtrar por todos los logs que provienen del archivo /var/log/mysql/error.log, que es el archivo donde MySQL guarda sus mensajes de error.

Y otra búsqueda con **{job="systemd-journal"} |= `error`**, para filtrar por todos los logs que provienen del **journald**, el sistema de logs de Linux que registra mensajes del sistema y de servicios y a su vez filtrar sólo las líneas que tengan la palabra “error”.

Figura 21. Filtrado de logs en Grafana.

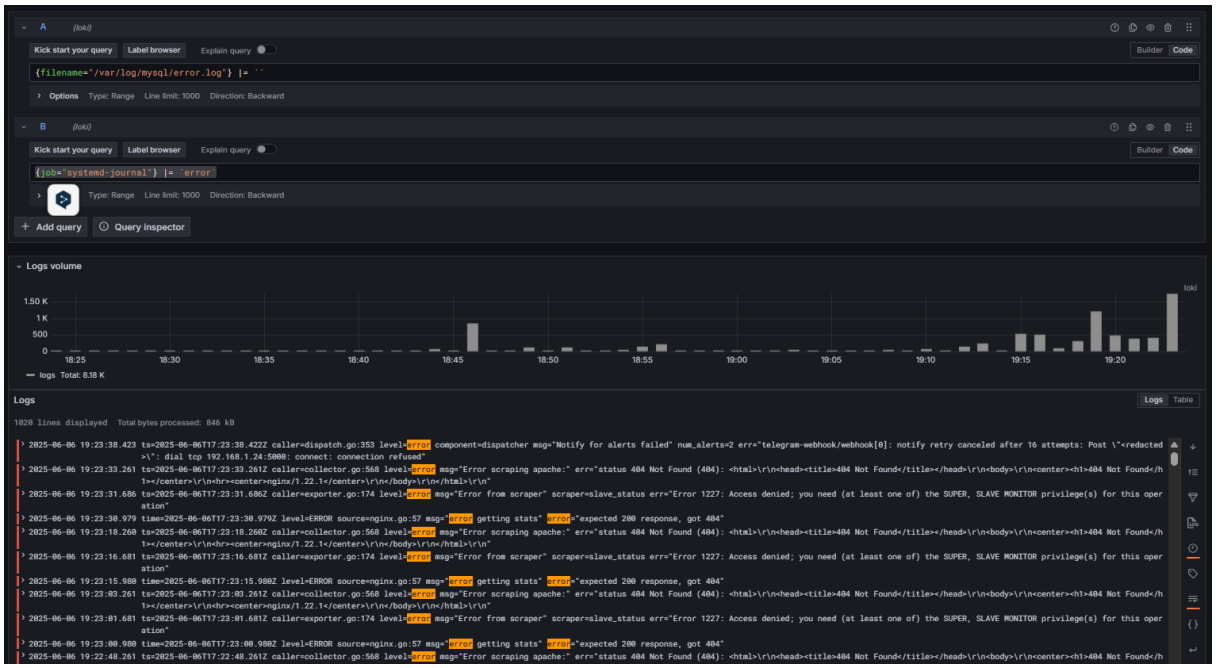

A (loki)
Kick start your query
Label browser
Explain query
Builder
Code
{filename="/var/log/mysql/error.log"} |= ``
Options Type: Range Line limit: 1000 Direction: Backward
B (loki)
Kick start your query
Label browser
Explain query
Builder
Code
{job="systemd-journal"} |= `error`
Type: Range Line limit: 1000 Direction: Backward
Add query
Query inspector
Logs volume
1.50 K
1 K
500
0
18:25
18:30
18:35
18:40
18:45
18:50
18:55
19:00
19:05
19:10
19:15
19:20
logs Total: 8.18 K
Logs
Logs
Table
1020 lines displayed Total bytes processed: 846 kB
2025-06-06 19:23:38.423 ts=2025-06-06T17:23:38.422Z caller=dispatch.go:353 level=error component=dispatcher msg="Notify for alerts failed" num_alerts=2 err="telegram-webhook/webhook[0]: notify retry canceled after 16 attempts: Post \"<redacted >\": dial tcp 192.168.1.24:5000: connect: connection refused"
2025-06-06 19:23:33.261 ts=2025-06-06T17:23:33.261Z caller=collector.go:568 level=error msg="Error scraping apache:" err="status 404 Not Found (404): <html>\r\n<head><title>404 Not Found</title></head>\r\n<body>\r\n <h1>404 Not Found</h 1> \r\n<hr> nginx/1.22.1 \r\n</body>\r\n</html>\r\n"
2025-06-06 19:23:31.686 ts=2025-06-06T17:23:31.686Z caller=exporter.go:174 level=error msg="Error from scraper" scraper=slave_status err="Error 1227: Access denied; you need (at least one of) the SUPER, SLAVE MONITOR privilege(s) for this oper ation"
2025-06-06 19:23:30.979 time=2025-06-06T17:23:30.979Z level=ERROR source=nginx.go:57 msg="error getting stats" error="expected 200 response, got 404"
2025-06-06 19:23:18.260 ts=2025-06-06T17:23:18.260Z caller=collector.go:568 level=error msg="Error scraping apache:" err="status 404 Not Found (404): <html>\r\n<head><title>404 Not Found</title></head>\r\n<body>\r\n <h1>404 Not Found</h 1> \r\n<hr> nginx/1.22.1 \r\n</body>\r\n</html>\r\n"
2025-06-06 19:23:16.681 ts=2025-06-06T17:23:16.681Z caller=exporter.go:174 level=error msg="Error from scraper" scraper=slave_status err="Error 1227: Access denied; you need (at least one of) the SUPER, SLAVE MONITOR privilege(s) for this oper ation"
2025-06-06 19:23:15.980 time=2025-06-06T17:23:15.980Z level=ERROR source=nginx.go:57 msg="error getting stats" error="expected 200 response, got 404"
2025-06-06 19:23:03.261 ts=2025-06-06T17:23:03.261Z caller=collector.go:568 level=error msg="Error scraping apache:" err="status 404 Not Found (404): <html>\r\n<head><title>404 Not Found</title></head>\r\n<body>\r\n <h1>404 Not Found</h 1> \r\n<hr> nginx/1.22.1 \r\n</body>\r\n</html>\r\n"
2025-06-06 19:23:01.681 ts=2025-06-06T17:23:01.681Z caller=exporter.go:174 level=error msg="Error from scraper" scraper=slave_status err="Error 1227: Access denied; you need (at least one of) the SUPER, SLAVE MONITOR privilege(s) for this oper ation"
2025-06-06 19:23:00.980 time=2025-06-06T17:23:00.980Z level=ERROR source=nginx.go:57 msg="error getting stats" error="expected 200 response, got 404"
2025-06-06 19:22:48.261 ts=2025-06-06T17:22:48.261Z caller=collector.go:568 level=error msg="Error scraping apache:" err="status 404 Not Found (404): <html>\r\n<head><title>404 Not Found</title></head>\r\n<body>\r\n <h1>404 Not Found</h</h

Figura 23. Alerta de mantenimiento en Telegram.

4. Prueba de alertas con Alertmanager + Telegram

Se configuraron alertas en Prometheus (ej. servicio caído, carga alta del sistema) y se simularon errores para forzar el envío de notificaciones.

Esas notificaciones en pocos segundos se recibieron en la aplicación Telegran en el canal previamente configurado.

Figura 24. Alertas configuradas en Prometheus.

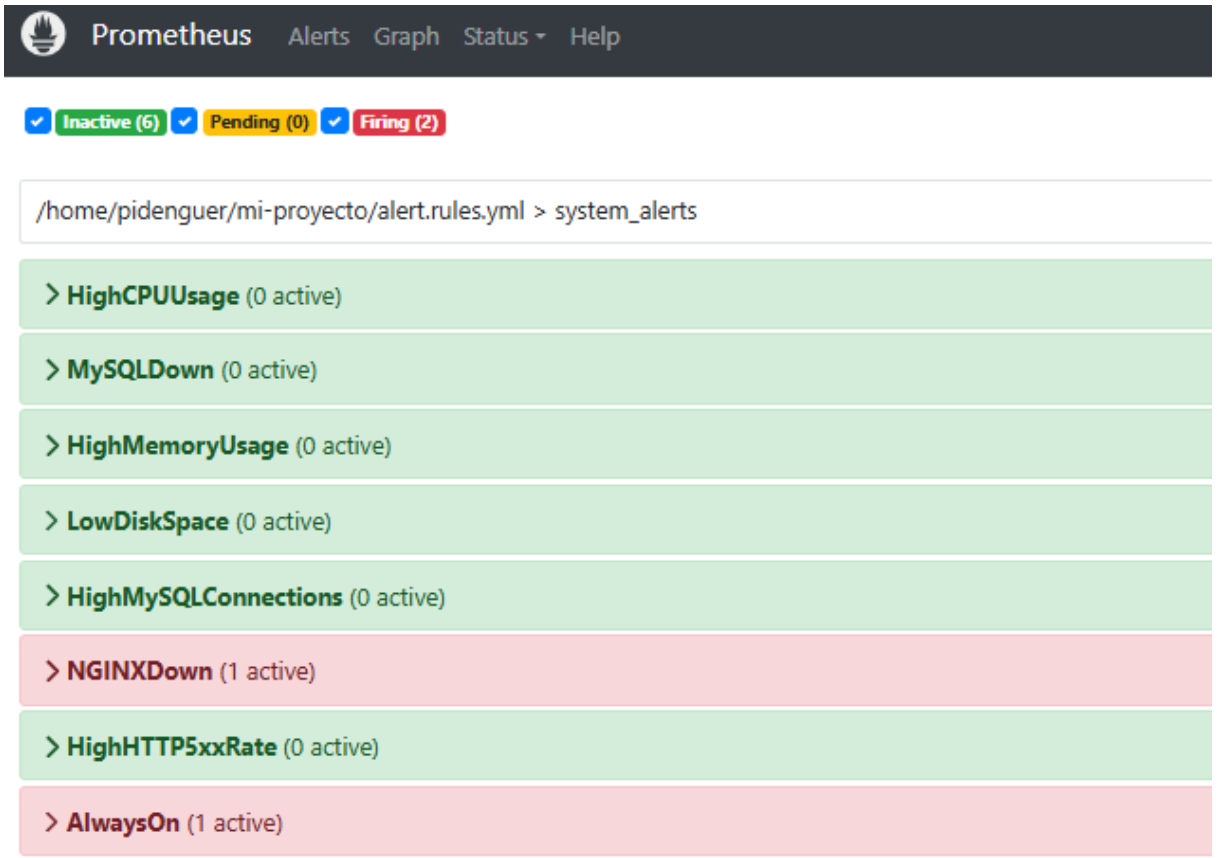
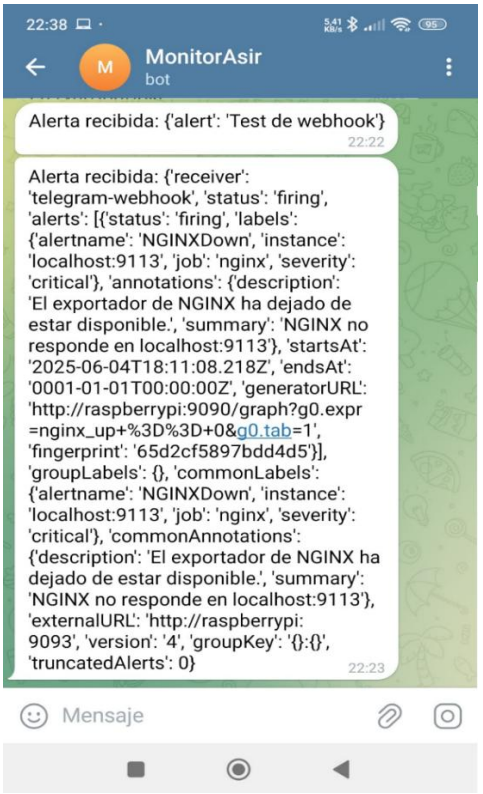


Figura 25. Alertas enviadas a Telegram.



2.5.2 Conclusiones de las pruebas

Las pruebas realizadas demuestran que el sistema es funcional y cumple con los objetivos propuestos:

- Monitoriza servicios y sistemas en tiempo real.
- Permite consultar logs de forma rápida y eficiente.
- Automatiza tareas básicas de mantenimiento.
- Notifica al usuario de problemas mediante Telegram de forma inmediata.

Con este sistema se reduce el tiempo de respuesta ante incidentes y se mejora la fiabilidad del entorno.

3. Conclusiones y trabajos futuros

3.1 Conclusiones.

A lo largo del desarrollo de este proyecto se ha logrado implementar un sistema completo de monitorización y mantenimiento automatizado, utilizando herramientas de software libre y de uso común utilizadas en el sector profesional, como Prometheus, Grafana, Loki, Alertmanager, y varios exporters.

Los objetivos principales del proyecto han sido:

- Se ha conseguido recopilar métricas detalladas de los servicios más críticos del sistema.
- Se ha desarrollado un script de mantenimiento automatizado que mejora la disponibilidad del sistema al reiniciar servicios caídos y eliminar logs antiguos.
- La visualización mediante dashboards en Grafana permite al administrador detectar problemas de forma clara y rápida.
- El uso de Alertmanager con integración a Telegram es un complemento muy util para recibir avisos en tiempo real ante incidencias.
- Se ha demostrado la utilidad de centralizar logs con Loki, mejorando la capacidad de análisis y diagnóstico.

Además, todo el sistema se ha instalado y configurado en una Raspberry Pi con IP estática, lo que confirma que se puede instalar un sistema de monitorización en entornos con recursos limitados.
Gracias a este proyecto se han podido poner en práctica conocimientos de redes, administración de sistemas, automatización de tareas con scripts, seguridad y uso de herramientas de observabilidad, todos ellos competencias clave dentro del perfil profesional de un técnico en ASIR.

3.2 Trabajos futuros.

Aunque el sistema actual cumple con los requisitos que se queerian llevar a cabo, existen líneas de mejora y ampliación que se podrían llevarse a cabo en el futuro:

- Automatización avanzada con Ansible: Implementar herramientas como Ansible para desplegar y configurar todo el sistema automáticamente en varios equipos.
- Copias de seguridad automáticas: Añadir scripts o herramientas de backup para proteger las configuraciones y los datos del sistema monitorizado.
- Monitorización de equipos remotos: Ampliar la solución para incluir varios equipos clientes (por ejemplo, con node_exporter) y visualizarlos desde un solo panel centralizado.
- Alertas más personalizadas: Configurar reglas más específicas en Prometheus, como consumo excesivo de recursos por procesos concretos o caídas de bases de datos.
- Gestión web del mantenimiento: Crear una pequeña interfaz web desde donde ejecutar tareas o ver el estado de los servicios monitorizados.
- Implementación de seguridad adicional: Aplicar autenticación más robusta en los paneles, cifrado de logs sensibles, o segmentación de red para servicios expuestos.

BIBLIOGRAFÍA

Christian Lempa. (2024, 31 de julio). *RaspberryPi 4 Monitoring Using Grafana and Prometheus* [Video]. YouTube. https://www.youtube.com/watch?v=RfucIfMNqsc&ab_channel=MBTECH

Prometheus Alertmanager Tutorial. (2020, 19 de noviembre). *Setting Up Alert Manager and Prometheus* [Video]. YouTube. https://www.youtube.com/watch?v=bPHOI29rwB0&ab_channel=Thetips4you

Linux Grafana Tutorial. (2020, 27 de julio). *Installing Grafana on a Raspberry Pi 4 Model B* [Video]. YouTube. https://www.youtube.com/watch?v=nA36o22OOGs&ab_channel=ThinkComputers

Prometheus-Grafana-AlertManager Monitoring. (2024). *Monitoring tutorial via Prometheus, Grafana and Alertmanager* [Playlist]. YouTube. https://www.youtube.com/playlist?list=PLI0vJRMEGNYSC28RVYPQjCDrCftu9_XcF

Luis Llamas. (2024). *How to Install Grafana on Raspberry Pi*. LuisLlamas.es.https://www.luisllamas.es/en/install-grafana-on-raspberry-pi/?utm_source=chatgpt.com

Prometheus Authors. (2024). *Prometheus Documentation*. Prometheus.io. https://prometheus.io/docs/introduction/overview/

Grafana Labs. (2024). *Grafana Documentation*. Grafana.com. https://grafana.com/docs/grafana/latest/

Grafana Labs. (2024). *Loki Documentation*. Grafana.com. https://grafana.com/docs/loki/latest/

Prometheus Community. (2024). *MySQL Server Exporter*. GitHub. https://github.com/prometheus/mysqld_exporter

NGINX Inc. (2024). *NGINX Prometheus Exporter*. GitHub. https://github.com/nginxinc/nginx-prometheus-exporter

Prometheus Authors. (2024). *Exporters and Integrations*. Prometheus.io. https://prometheus.io/docs/instrumenting/exporters/